Specification of the Sonnet Programming Language

Version 0.1.0 dated 03.07.2026

StefanValentinT

Abstract

This specification details the syntactic form and evaluation of programs written in the Sonnet programming language. It details, how programs are represented, how these representations map to an understnading of terms, and how these terms get evaluated. A formalization of these necessarily informal descriptions is also provided within this specification. In doing so it defines the requirements for a conforming implementation, thus establishing a single source of truth for the meaning of any program to promote the portability of the language itself.

Contents

1. Introduction

Sonnet ought to be beautiful in its simplicity, pristine in its architecture, bright-shining as the ivory tower, with a long spiral staircase winding its way all the way to the top, twisting round and round the tower. Sonnet shall be there for everyone, shall be easy to use because of its simplictly, but limitless in its reach. Sonnet shall bring back the simple, arcane joy of programming.

Therefore, the specification is written in a way to facilitate both unambiguity and accessibility for everyone. Whenever these goals conflicted, it was opted to add important information using a Note, or, where needed, to provide additional reiteration or exemplification marked with an Example, whilst not compromising the precise actual specification. Both shall be regarded as non-authoritative for the meaning of the authorative text and may only aid in its understanding.

1.1. Scope

First the actual representation of Sonnet programs is detailed, before specifying their syntactical structure. Thereafter the constraints and requirements enacted upon the maening of a program are detailed and its evaluation via the reduction of terms. The means by

which a program may interface with the surrounding world are described and lastly a collection of definitions is given that a conforming implementation shall provide as minimal, opaque "atoms" of computation.

1.2. Limitations

No bound is given for the complexity of a program that may exceed the capabilities of a particular evaluator, compiler or processor. Although such limits seem to be necessary in practice, especially for data, neither the maximal complexity of any allowed program nor the minimal abilities of a system are mandated, because doing so would constrain the language and its implementations by artificial limits.

Optimization. The means by which a conforming evaluator is invoked or the way it evaluates a given program are left unspecified to allow for greater freedom of optimization. Only directly observable behaviour of the language itself shall be specified.

2. Conformance & Specification

The following definitions of verbs shall only apply within the context of a constraint on the implementation.

The use of "shall" denotes a mandatory requirement on an implementation, while "shall not" and "may not" declare a strict prohibition.

"May" is used to specify that some behaviour is conformant to the specification, noticeably it is not exclusive, signifying multiple interpretations may be within the bounds of conformance. "Need not" is used to explicitly denote the non-requiredness of a particular property.

A "conforming" implementation is any one that abides to all requirements laid out by this specification, including those requiring the rejection of invalid or erroneous programs. Similarily, a conforming program is one not rejected by a conforming implementation.

A implementation shall be accompanied by a clear definition of the behaviour in all cases of implementation-defined behaviour in this specification, for all cases where the characteristics do not match the recommendations of this specifications non-authorative text.

3. Definitions

The following definitions apply to the specification. Other terms are defined when they first occur or in a more appropriate section; when a term is defined it is being written in *italic* type.

- Observable **behaviour** is all action that influences the world. Observable influence may also be called physical. Non-observable behaviour does not influence the world and may be assumed to not exist in a program.
- **Bit** is a unit of data storage capable of holding one of two values, commonly referred to as $\{0, 1\}$. Individual bits do not need to be addressable.
- A **byte** is the smallest unit of bits that is addressable.

  > Note A byte commonly consists of 8 bits though a different length can be used for a conforming implementation.

- **Binary data** is the representation of data using bits, consisting of one or more bytes.
- A **character** is a member of a set of characters that make up the textual representation of a Sonnet program.
- An **implementation-defined** value or behaviour is one for which the specification provides multiple possibilites and imposes no further requirements on which is chosen in any instance where each implementation must document its choice. A subset of this is **locale-defined** where rather than the implementation the circumstances of its invocation define certain behaviours.

> Example The behaviour of functions such as `is_lower` may depend on the language a user has set for their computer.

– **Unspecified** behaviour is behaviour for which the specification provides multiple possibilites and imposes no further requirements on which is chosen in any instance. After the unspecified behaviour, program execution continues normally.

> Example The evaluation order of + is unspecified; thus
>
> ```
> print("Hello ") + print("world!")
> ```
>
> may produce either “Hello World!” or “world!Hello “.

– **Undefined** behaviour is behaviour for which this specification imposes no requirements, it being the result of evaluating erroneous program constructs or data. Any subsequent deviations in the execution of a program are causal results of the specific outcome triggered by the evaluation leading to undefined behaviour. Prior to the manifestation of the undefined operation, all observable behaviour appears as specified in the execution of the program.

4. Conventions

– The term *value set* is used to refer to the set of representable values for some type.

– All instances of unspecified, undefined or implementation-specified behaviour are explicitly marked with the corresponding adjective.

5. Syntax

5.1. Textual representation

The representation form of Sonnet programs is text; a program is a sequence of characters. The set of characters must at least suffice the following requirements, being able to encode all listed characters:

– all the letters from the Latin alphabet, those being:

A B C D E F G H I J K L M
N O P Q R S T U V W X Y Z

– and their lowercase equivalents:

a b c d e f g h i j k l m
n o p q r s t u v w x y z

– all arabic digits:

0 1 2 3 4 5 6 7 8 9

– as well as the following set of special characters (“␣” denotes the invisible space character):

\+ - * / < > = ! ? & | % $
. : ; , ( ) [ ] { } ~ _ ␣

The encoding of a program except for all string literals it might contain, is implementation-defined.

> Note At the time of writing (2026) it seems best to encode the whole file in UTF-8 unless special domain constraints would apply.

Implementations may allow to substitute Unicode signs such as “→” for their normal counterparts (e. g. “->”).

String literals shall be represented as text encoded in the UTF-8 format.

Additonally a character or sequence of character shall exist, to encode a line break; in the following chapters this will be represented by the sequence “\n”.

5.2. Definition of the syntax notation

A terminal is a sequence of characters, a non-terminal is a variable, with at least one expansion. An item is either a terminal or a non-terminal. Whitespace is insignificant and not a terminal.

Terminals are written in **bold** type, whilst non-terminals are written in normal type.

A production rule is defined by a non-terminal on the left followed by an arrow and a expansion on the right. It may be read as “non-terminal x may expand into *expansion*”.

$$
\begin{array}{rcrl}
\iota & := & \text{i8} \mid \text{i16} \mid \text{i32} \mid \text{i64} & \text{(signed integer types)} \\
& \mid & \text{u8} \mid \text{u16} \mid \text{u32} \mid \text{u64} & \text{(unsigned integer types)} \\
& \mid & \text{f16} \mid \text{f32} \mid \text{f64} & \text{(floating-point types)} \\
\\
\tau & := & \iota & \text{(primitive types)} \\
& \mid & \alpha & \text{(type variable)} \\
& \mid & \forall \alpha.\tau & \text{(universally quantified type)} \\
& \mid & \tau \wedge \tau & \text{(intersection)} \\
& \mid & \tau^* \to \tau & \text{(function type)} \\
& \mid & *\,\tau & \text{(pointer type)} \\
& \mid & [\tau, n] & \text{(array type)} \\
& \mid & \text{struct}(\varphi) & \text{(anonymous struct type)} \\
& \mid & \text{union}(\varphi) & \text{(anonymous union type)} \\
\\
\varphi & := & (x, \tau)^* &
\end{array}
$$

Figure 1: The abstract syntax of types in Sonnet.

Optional items are enclosed within braces. If the brace pair is prepended with a "+" the enclosed item shall appear at least once, with no upper bound on the amount of repetitions, if prepended with a "*" the item may appear zero or more times.

Alternation is defined using the vertical bar as in " $a \mid b$"), which reads "expand into either a or b". Similarily, "one of" expands into out of a list of terminals.

The expansion process of production rules terminates successfully if and only if the input sequence is syntactically correct. Upon termination, no further production rules can be applied, and the resulting string consists exclusively of terminal symbols.

5.3. The Syntax

6. Abstract Syntax

The abstract syntax describes the structure of a Sonnet program independent of its textual representation. It is presented in a similiar fashion as inductive datatypes in most programming languages. This naturally maps to their modelling within Lean in Section 9. In the grammar, a^* denotes that a may appear once or multiple times. x represents an identifier. The set of all possible expansions of a non-terminal x is written as $\varepsilon(x)$.

6.1. Types

The abstract syntax for types is given in Figure 1. The set of types of some rank n T_n is a subset of the types derived by expansion of τ as defined in this grammar. The function type constructor $\to$ associates to the right and binds weaker than all other type constructors, both $t_1 ... t_n \to \sigma$ and $\forall t.\sigma$ extend rightward to the end of the expression within they occur, meaning $t \to t \to t \wedge t$ is equal to $t \to (t \to (t \wedge t))$ and $\forall t.t \wedge t$ is equal to $\forall t.(t \wedge t)$.

Rank. A type is of rank n with respect to a specific syntactic construct if every path from the root of the type tree to that construct traverses the left branch of fewer than n function type constructors.

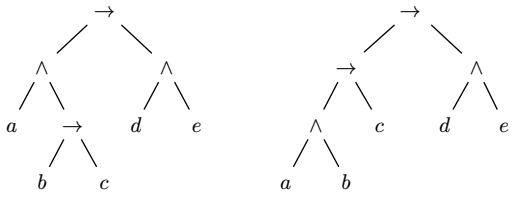

Figure 2: A rank-2 intersection type on the left and a non-rank-2 type on the right.

Types of rank 0 are also called "simple types". T_2, the set of rank-2 intersection types, is defined inductively:

$$T_0^0 = \varepsilon(\alpha) = T_1^0 = T_2^0$$

For each natural number i, the sets T_0^{i+1} and T_1^{i+1} and T_2^{i+1} are defined as follows:

$$T_0^{i+1} = T_0^0 \cup \left\{(\sigma_1 ... \sigma_n \to \tau) \mid \forall j.\sigma_j \in T_0^i, \tau \in T_0^i\right\}$$

$$T_1^{i+1} = T_0^{i+1} \cup \left\{(\sigma \wedge \tau) \mid \sigma, \tau \in T_1^i\right\}$$

$$T_2^{i+1} = T_1^{i+1} \cup \left\{(\sigma_1 ... \sigma_n \to \tau) \mid \forall j.\sigma_j \in T_1^i, \tau \in T_2^i\right\}$$

T_2 is the union of all rank-2 construction stages.

$$T_2 = \bigcup_i T_2^i$$

Finally, let the complete set of types be T:

$$T = T_2 \cup \{(\forall t.\sigma) \mid \sigma \in T, t \in \varepsilon(\alpha)\}$$

As is evident from the definition, quantification can only appear at the top-level of a type.

Since $\wedge$ is commutative and idempotent ($\alpha \wedge \alpha = \alpha$) any rank-1 type may also be written as the intersection over a non-empty, finite set of simple types: $\bigcup \bigwedge S$ where $S \subseteq T_0$ and $S \neq \emptyset$.

Example It is shown that the rank-2 type from Figure 2 is in fact contained within T:

By definition, all type variables belong to T_0^0.

Using the (reduced) definition for T_0^{i+1}:

$$T_0^1 = T_0^0 \cup \left\{(\sigma \to \tau) \mid \sigma, \tau \in T_0^0\right\}$$

From $b, c \in T_0^0$, it follows that $(b \to c) \in T_0^1$

From $T_0^0 \subset T_0^1$, it follows that $a, d, e \in T_0^1$

Using the definition of T_1^1:

$$T_1^1 = T_0^1 \cup \left\{(\sigma \wedge \tau) \mid \sigma, \tau \in T_1^0\right\}$$

Because $T_1^0 = T_0^0$, it follows $(d \wedge e) \in T_1^1$

Using the definition of T_1^2:

$$T_1^2 = T_0^2 \cup \left\{(\sigma \wedge \tau) \mid \sigma, \tau \in T_1^1\right\}$$

Since $a \in T_0^0 \subset T_1^1$ and $(b \to c) \in T_0^0 \subset T_1^1$, it follows that $(a \wedge (b \to c)) \in T_1^2$.

Using the definition for T_2^3:

$$T_2^3 = T_1^3 \cup \left\{(\sigma \to \tau) \mid \sigma \in T_1^2, \tau \in T_2^2\right\}$$

Since it has been established that $(a \wedge (b \to c)) \in T_1^2$ and $(d \wedge e) \in T_1^1 \subseteq T_1^2 \subseteq T_2^2$.

Therefore, by choosing these elements for σ and τ, the type is obtained:

$$((a \wedge (b \to c)) \to (d \wedge e)) \in T_2^3$$

Since $T_2^3 \subseteq T_2$ by definition of T_2 and $T_2 \subseteq T$ it has been shown that the type is contained within T.

6.2. Terms

The abstract syntax for terms is given in Figure 3. A term is any construct that can be derived by expansion of t as defined in this grammar.

7. Typing

As a consequence of Rice's Theorem the type system can only be a syntactic mechanism to approximate a programs semantics [1]. Thus it has been designed to allow the programmar great freedom in the way a program is written

$$\eta \;::=\; x \mid \eta : \tau$$

$$\begin{array}{rcl}
t & ::= & \\
 & & x \qquad\qquad\qquad\qquad \text{(variable)} \\
 & \mid & (n, \iota) \qquad\qquad\quad \text{(typed literals, } n \in \mathbb{Q}\text{)} \\
 & \mid & t : \tau \qquad\qquad\qquad \text{(annotation)} \\
 & \mid & \eta^* \to t \qquad\qquad\quad \text{(function terms)} \\
 & \mid & x(t^*) \qquad\qquad\qquad \text{(function application)} \\
 & \mid & \{t^*\} \qquad\qquad\qquad \text{(data literal)} \\
 & \mid & \text{if } t \text{ then } t \text{ else } t \qquad \text{(conditional branching)} \\
 & \mid & \text{while } t \text{ do } t \qquad\quad \text{(loop)} \\
 & \mid & \text{return } t \qquad\qquad\quad \text{(function exit)} \\
 & \mid & \text{block } t^* \qquad\qquad\quad \text{()}
\end{array}$$

Figure 3: The abstract syntax of terms.

$$\text{(Var)} \quad \frac{\Gamma \cup \left\{x : \bigwedge_{i \in I} \tau_i\right\} \quad i_0 \in I}{\Gamma \vdash x : \tau_{i_0}}$$

$$\text{(ABS)} \quad \frac{\Gamma \cup \{x_1 : \sigma_1, ..., x_n : \sigma_n\} \vdash t : \tau \quad \text{where } \sigma_i = \begin{cases} \tau_i & \text{if } \eta_i = (x_i : \tau_i) \\ \alpha_i & \text{if } \eta_i = x_i \end{cases}}{A \vdash (\eta_1, ..., \eta_n \to t) : (\sigma_1 ... \sigma_n) \to \tau}$$

Figure 4: The declarative typing rules of Sonnet.

within the bounds of making both type checking and type inference sound and decidable.

All terms have a type, written as $t : \tau$. The context in which evaluation and typing occurs is written Γ, thus $\Gamma \vdash e : \tau$ reads as "e has type τ in context Γ".

A type environment, denoted by Γ, is a relation between a set of variables and a set of types, defined as a finite set of distinct variable-type pairs $\{x_1 : \sigma_1, ..., x_n : \sigma_n\}$ that tracks the types of variables currently in scope. The domain of a type environment, written $\text{dom}(\Gamma)$, is the set of all variable names present in the relation being $\{x \mid \exists \sigma.(x : \sigma) \in \Gamma\}$. For any variable $x \in \text{dom}(\Gamma)$, $\Gamma(x) = \sigma$ is defined such that $(x : \sigma) \in \Gamma$.

A typing judgment $\Gamma \vdash t : \tau$ asserts that a term t is of type τ in the type environment Γ.

The typing rules are given in Figure 4. The types used in the position of premises are of rank 1, whereas those derived are of rank 2.

8. Semantics

8.1. Numeric Types

The numeric types are the only primitive types. They are `i8`, `i16`, `i32`, `i64`, `u8`, `u16`, `u32`, `u64`, `f16`, `f32` and `f64`. The number types of the form in, un or fn where $n \in \mathbb{N}$ use n bits for the representation of a value of this type. n is a multiply of bit-length of a byte.

More primitive number types may be defined by an implementation.

Signed integer types. The four signed integer types are `i8`, `i16`, `i32` and `i64`.

A signed integer type in represents a value encoded using a two's-complement binary representation consisting of n bits. The set of representable values $V_{\text{i}n}$ for a type in is:

$$V_{\text{i}n} = \{x \in \mathbb{Z} \mid -2^{n-1} \leq x \leq 2^{n-1} - 1\}$$

Unsigned integer types. The four unsigned integer types are `u8`, `u16`, `u32` and `u64`.

An unsigned integer type un represents a value encoded as a standard binary number. The set of representable values $V_{\text{u}n}$ for a type un is:

$$V_{\text{u}n} = \{x \in \mathbb{Z} \mid 0 \leq x \leq 2^n - 1\}$$

Floating-point types. The three floating-point types are f16, f32 and f64. A floating-point type fn represents a value encoded using the IEEE 754 standard for floating-point arithmetic [2].

Correspondence between floating-point types and the formats defined by IEEE 754:

- `f16` half-precision floating-point
- `f32` single-precision floating-point
- `f64` double-precision floating-point

8.2. Conversion between number types

Conversion between integer types. The conversion of an integer type to a larger integer type, where the target value set is a superset of the source value set, preserves the original value.

Integer to integer of equal size. The set of nonnegative values inhabiting a signed integer type is a subset of the set for the corresponding unsigned integer type, and the representation of the same value shall be identical in both types.

A negative integer a is converted to a positive unsigned integer by adding 2^n where n is the number of bits used to represent a. Conversely, converting an unsigned integer to a signed integer is done by subtracting 2^n.

> NOTE A signed value -2^{n-1} becomes 2^{n-1}, the middle of the range of integers of the correspondent unsigned type. The biggest value representable by an unsigned type, $(2^n - 1)$, become -1.

Unsigned integer to larger integer. The value remains preserved exactly in accordance to Section 8.2.1.

Signed integer to larger integer. A signed integer value a is converted to a larger target integer type of n bits by preserving its value exactly if the target type is signed or if $a \geq 0$. If $a < 0$ and the target type is unsigned, the value is converted by adding 2^n.

Integer to integer of lesser size. An integer value a is converted to a smaller target integer type of n bits by first reducing its value modulo 2^n to an integer in the range $[0, 2^n - 1]$. If the target type is signed and the resulting value is greater than or equal to 2^{n-1}, the final value is obtained by subtracting 2^n from that result.

> NOTE Operationally, this corresponds to truncating the higher-order bits, retaining only the lowest n bits of the original binary representation.
>
> If the target type is unsigned, these remaining bits are interpreted as a standard binary number. If the target type is signed, the highest bit of the remaining n bits acts as the sign bit.

Floating-point number to integer. If the floating-point numbers' integral component can be represented in the target type that shall be the result, otherwise the result is implementation-defined.

Integer to floating-point. The result of converting a value of an integer type (in or un) to a floating-point type fm is determined by whether the value can be exactly represented in the target format:

- If the integer value can be represented exactly in the target floating-point type, the result is that exact representation.
- If the integer value cannot be represented exactly but falls between two representable floating-point values, the value is rounded to one of the adjacent representable values. The choice of rounding direction is implementation-defined but must conform to the IEEE 754 standard.
- If the integer value exceeds the maximum representable finite value of the target floating-point type, the result is undefined.

Floating-point to floating-point. The result of converting a value of a floating-point type fn to another floating-point type fm depends on the relative precision of the source and target formats:

- If the source value can be represented exactly in the target format, the value shall remian unchanged.
- If the source value cannot be represented exactly because the target format has less precision, the value is rounded to one of the adjacent representable values. The rounding direction is implementation-defined but must conform to the IEEE 754 rounding modes.
- If the magnitude of the source value exceeds the maximum representable finite value of the target type, the result is implementation-defined.

Integer Overflow. If an integer by means of any operation on it overflows the resulting value is implementation-defined.

9. Concrete Semantics

To eliminate ambiguity in the interpretation of the semantics presented above, they have been formalized in the Lean proof assistant. The complete formalization is included as part of this specification. Wherever possible, references to the corresponding informal definitions are provided.

The formalization was carried out via a reference implementation that evaluates an abstract syntactic construct of a Sonnet program to the set of all its defined values and behaviours, thereby encompassing every value and every behaviour the construct may yield in a conforming implementation.

The type definitions of Figure 1 in Lean.

```
-- primitive types
inductive PrimitiveType : Type
  | i8  : PrimitiveType
  | i16 : PrimitiveType
  | i32 : PrimitiveType
  | i64 : PrimitiveType
  | u8  : PrimitiveType
  | u16 : PrimitiveType
  | u32 : PrimitiveType
  | u64 : PrimitiveType
  | f16 : PrimitiveType
  | f32 : PrimitiveType
  | f64  : PrimitiveType
  deriving Repr

-- α
structure TypeVar where
  name : String
  deriving Repr, DecidableEq, Hashable

inductive SimpleType : Type where
  | var : TypeVar → SimpleType
  | arrow : SimpleType → SimpleType → SimpleType
  deriving Repr, DecidableEq

inductive NonEmptyList (α : Type)
  | mk : α → List α → NonEmptyList α
  deriving Repr, DecidableEq

structure IntersectionType where
  components : NonEmptyList SimpleType
  deriving Repr

inductive Rank2Type : Type where
  | simple : SimpleType → Rank2Type
  | inter : IntersectionType → Rank2Type
  | arrow : IntersectionType → Rank2Type → Rank2Type
  deriving Repr
```

References

[1] H. G. Rice, "Classes of recursively enumerable sets and their decision problems," *Transactions of the American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953, doi: 10.1090/S0002-9947-1953-0053041-6.

[2] "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, no. , pp. 1–84, 2019, doi: 10.1109/IEEESTD.2019.8766229.